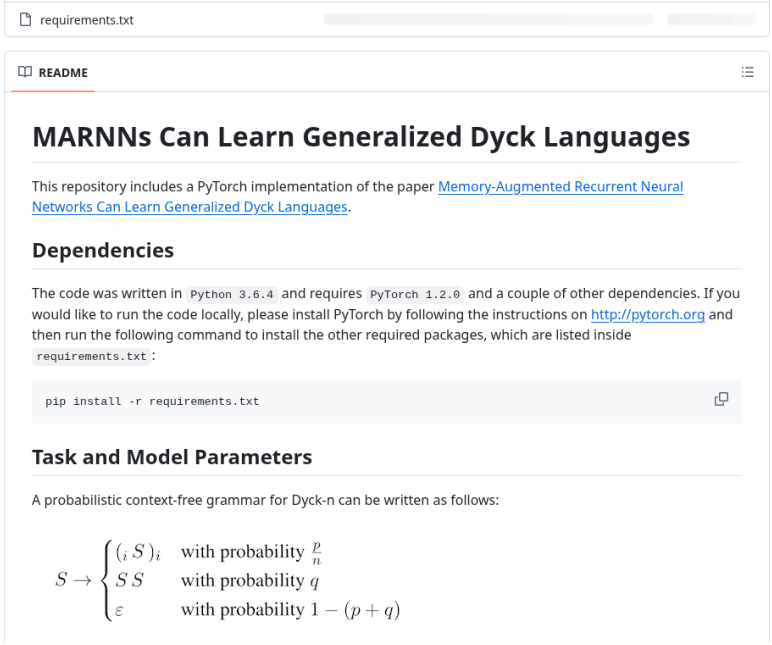
Dyck Language

括弧の入れ子構造を補完する

sasakiy84

著者の公式実装



requirements.txt

README

MARNNs Can Learn Generalized Dyck Languages

This repository includes a PyTorch implementation of the paper Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages.

Dependencies

The code was written in `Python 3.6.4` and requires `PyTorch 1.2.0` and a couple of other dependencies. If you would like to run the code locally, please install PyTorch by following the instructions on http://pytorch.org and then run the following command to install the other required packages, which are listed inside `requirements.txt`:

```
pip install -r requirements.txt
```

Task and Model Parameters

A probabilistic context-free grammar for Dyck-n can be written as follows:

$$S \rightarrow \begin{cases} (_i\, S\,)_i & \text{with probability } \frac{p}{n} \\ S\, S & \text{with probability } q \\ \varepsilon & \text{with probability } 1-(p+q) \end{cases}$$

[1] Suzgun et al. MARNNs Can Learn Generalized Dyck Languages. 著者の公式実装。 https://github.com/suzgunmirac/marnns

1. 括弧列の補完と実例
2. HELM の採点と公開成績
3. 生成方法とデータの構成
4. 研究での使われ方と入手

括弧列の補完と実例

Dyck 言語は、正しく対応する括弧の列

- Dyck-n は、 n 種類の括弧対を使い、開く括弧と閉じる括弧が正しく入れ子になる列の集合。
- HELM の公開設定は丸括弧 ( ) 、角括弧 [ ] 、波括弧 { } の 3 種類を使う。
- 入力は閉じ括弧が欠けた列。開いたままの括弧を閉じる、最短の続きの列を生成する。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

数だけでなく、開いた順序を記憶する

- 閉じる括弧の種類は、最後に開いたままの括弧で決まる。単に種類ごとの個数を数えるだけでは順序を復元できない。
- スタックは、最後に積んだ要素を最初に取り出す記憶の仕組み。開く括弧を積み、閉じるたびに対応を照合する。
- 以下の実例は固定 seed の公式 HELM 生成器から得た実データで、公開 run の入力とも照合した。

[1] Suzgun et al. (2019). Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages. arXiv. https://arxiv.org/abs/1911.03329

[2] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例１：３つの閉じ括弧で完成する

入力の全文／ 65 記号。空白で区切り、表示上の改行だけを追加。

```
( [ ( ) ( { { ( ( { { } } ) ) } { [ ( { } (
[ ] ) [ { } ] ) ] } } [ ( { ( ( [ [ [ [ [ ]
] ] ] ] ) ) } ) ] { [ ] } { ( ( ) ) } ( )
```

正解：） ］ ）

開いたままの括弧を、内側から順に閉じる。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id3 / test / generator index 3
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例２：括弧の種類と順序を保つ

入力の全文／ 89 記号。空白で区切り、表示上の改行だけを追加。

```
[ ] { [ ] ( { ( [ [ { } ] ] ) } ) [ ( [ ( ( [
] { [ ( ) ( ) ] } ) ) ] [ { } ] ) { ( ( { [ (
) ( { } [ { [ [ [ [ ( ( ) ( ) [ [ ] ] ) ] ] ]
{ [ ] } ] } ] [ ( ) ] ) ] } ) [ [ ] ] )
```

正解：}] }

開いたままの括弧を、内側から順に閉じる。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id4 / test / generator index 4
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例３：長い列から必要な括弧を取り出す

入力の全文／ 63 記号。空白で区切り、表示上の改行だけを追加。

```
{ [ [ { ( { } ) } ] ] } [ ( { ( ( { ( { [
( [ [ ( { ( ) } ) [ [ ] ] ] { ( [ ] [ ] )
} ] [ { [ [ ] ] } ] ) ] } ) } ) ( ( ) ) )
```

正解： **})]**

開いたままの括弧を、内側から順に閉じる。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id9 / test / generator index 9
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

正解は、残ったスタックを逆順に閉じる列

- 実例 1 で開いたままなのは、外側から丸括弧、角括弧、丸括弧の順。内側から閉じる正解は「)]) 」。
- 正解に開く括弧は含めない。対応する閉じ括弧だけを、残った数だけ出す。
- 生成器は正解をスタックから計算する。人が注釈して複数の回答を集める課題ではない。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

HELM の採点と公開成績

英語の指示と 3 件の完成例を提示する

公開 run の指示全文と日本語訳

> Please complete the rest of the following Dyck sequences, making sure that the parentheses are closed properly.

以下の Dyck 列の残りを補完し、括弧が正しく閉じられるようにしてください。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

公開指標は、最後の空白区切り要素を比べる

- HELM の設定は exact_match_indicator を指定する。実装は予測と正解を空白で分け、最後の要素だけを取り出して比較する。
- 最後の要素が一致すれば 1 点、異なれば 0 点。 500 件の得点を平均する。
- 正解列のすべてが合うかを測る、列全体の完全一致とは異なる。

[1] HELM v0.3.0. exact_match_indicator. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

余分な括弧があっても、末尾は一致する

実例 3 の公開予測： text-davinci-003 、試行 0

正解： })]

予測：] })]

最後の「]」は一致するため公開指標は 1 点。余分な「]」が先頭にあるため、列全体の完全一致は 0 点になる。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003
[2] HELM v0.3.0. exact_match_indicator. id9 / train_trial_index=0 https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

HELM の公開得点：末尾要素の一致率

モデル	一致率
Anthropic-LM (2022)	84.87%
text-davinci-003 (2022)	75.07%
OPT-175B (2022)	49.40%

test 500 件、各 3 試行の平均。 Classic v0.4.0 掲載、 run suite v0.3.0 。

[1] HELM Classic v0.4.0, Dyck Language. 公開 run suite v0.3.0 、括弧 3 種類。 https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

同じ公開出力を、列全体でも再採点する

text-davinci-003	末尾要素	列全体
試行 0	73.2%	40.6%
試行 1	76.0%	49.2%
試行 2	76.0%	49.2%
3 試行の平均	75.07%	46.33%

同じ 1,500 出力を再集計。列全体は前後の空白を除いた文字列を比較する。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

指標の実装は末尾要素だけを取り出す

```
pred = pred.split(indicator)[-1].strip()
gold = gold.split(indicator)[-1].strip()
return exact_match(gold, pred)
```

indicator の既定値は半角空白。最後の要素より前にある誤りは、この指標では採点しない。

[1] HELM v0.3.0. exact_match_indicator. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

測りたい能力に、採点を対応させる

- 課題の目標は、開いた括弧を正しい順序ですべて閉じること。列全体の正しさは、出力の途中にある誤りや余分な括弧にも依存する。
- 公開得点は末尾要素の一致率として解釈し、補完能力を調べるときは出力全体も照合する。
- これらは 2023 年に公開された HELM の実行結果であり、2026 年のモデル順位ではない。

[1] HELM v0.3.0. exact_match_indicator. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py
[2] HELM Classic v0.4.0, Dyck Language. 公開 run suite v0.3.0 、括弧 3 種類。 https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

生成方法とデータの構成

確率的な文法で括弧列を生成する

生成規則	確率
S → 開く括弧 i S 閉じる括弧 i	p/n （各種類）
S → S S	q
S → 空列	1 − p − q

HELM は n=3 、 p=0.5 、 q=0.25 。入れ子と連結を繰り返す。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

完成列を作り、補完すべき短い列を求める

- 文法から完成した括弧列を生成し、指定した長さの範囲に入る候補を採る。
- 完成列の後ろから走査し、残す入力を切り出す。切り落とす側には、対応済みの部分列が含まれる場合もある。
- 最後に残した入力をスタックで読み直し、必要な閉じ括弧を計算する。正解の長さは１〜３記号になる。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

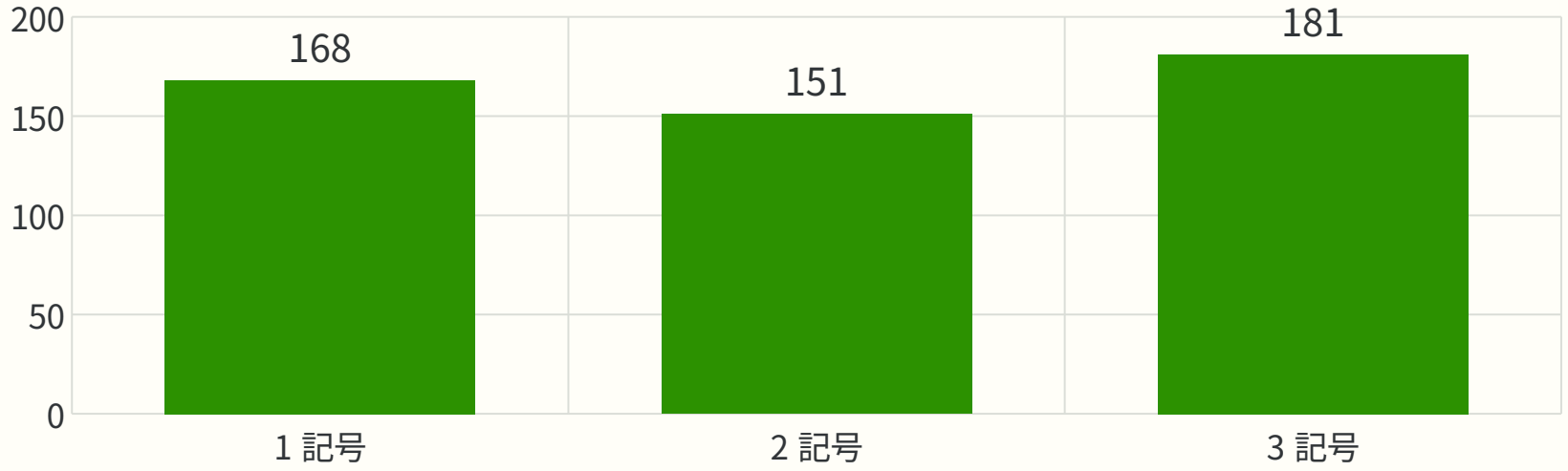
提示例 3 件と、評価例 500 件を作る

設定	提示例用	評価用
件数	3	500
切り出す前の生成列長	4 〜 50 記号	52 〜 100 記号
正解の最大長	3 記号	3 記号

seed=42 。入力文字列は集合内で重複させず、提示例用と評価用の重複も除く。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

正解の長さは 1 〜 3 記号に分かれる



固定 seed で生成した評価 500 件の全件集計。縦軸は件数。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

評価入力は 45 〜 99 記号、中央値 71 記号

量	固定 seed の 500 件
入力の最小／最大	45 ／ 99 記号
入力の中央値	71 記号
入力の平均	71.418 記号
最大入れ子深さの最小／最大	6 ／ 30

空白で区切った括弧記号を数えた。生成列から切り出した後の入力を集計。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

seed と実装を固定し、公開入力と照合する

- 公式シナリオは Python の random と NumPy の乱数を、どちらも 42 で初期化する。
- v0.3.0 の生成処理で 503 件を再現し、評価 500 件すべての入力と正解が公開 run と一致することを確認した。
- 各例には生成時の位置、 ID 、原レコードと SHA-256 を保存した。閉じ括弧の正解は別のスタック処理でも全件検証した。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

研究での使われ方と入手

2019 年の研究は、各時点の次記号を予測する

- Suzgun らは、外部メモリを持つ再帰型ニューラルネットが、入れ子構造を持つ Dyck 言語を学べるか調べた。
- 各時点で、それまでの列に続けられる記号の集合を予測する。全時点の予測が正しい列を成功と数えた。
- HELM の「末尾の閉じ括弧を生成する課題」とは、出力の形式も採点の単位も異なる。

[1] Suzgun et al. (2019). Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages. arXiv. https://arxiv.org/abs/1911.03329

Dyck-2 の研究では、外部メモリが役立つ

2019 年の方式	テスト成功率の平均
Vanilla RNN	0.00%
Vanilla LSTM	1.39%
Stack-RNN + Softmax	99.99%

10 回の実験。学習 5,000 列（2 〜 50 記号）、テスト 5,000 列（52 〜 100 記号）。

[1] Suzgun et al. (2019). Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages. arXiv. https://arxiv.org/abs/1911.03329

BIG-bench と HELM は、補完形式が異なる

対象	データと出力
BIG-bench	Dyck-4 、固定 1,000 例、選択肢の得点で評価
HELM の公開設定	Dyck-3 、 seed 固定、閉じ括弧列を生成

BIG-bench は選択肢を入力へ追記しない設定。 HELM は独自の生成器を使う。

[1] BIG-bench. Dyck Language (2026-10-18 取得) 。 https://github.com/google/BIG-bench/tree/main/bigbench/benchmark_tasks/dyck_languages

生成器と採点コードを一緒に選ぶ

- HELM v0.3.0 の DyckLanguageScenario で、括弧の種類数、件数、長さ、 seed を指定できる。生成時に外部データの取得は要らない。
- 公式 HELM 実装のコードは Apache License 2.0 。元研究の実装と、 HELM の生成・評価コードは別の配布物である。
- 2026-10-18 時点でも、著者の実装、 BIG-bench の固定例、HELM の公開 run を参照できる。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py
[2] BIG-bench. Dyck Language (2026-10-18 取得) 。 https://github.com/google/BIG-bench/tree/main/bigbench/benchmark_tasks/dyck_languages

括弧の順序を覚える課題と、採点の範囲

- Dyck 補完は、未完の括弧を内側から正しく閉じる課題。正解はスタックから一意に計算できる。
- HELM では短い提示例から長い評価列へ移る。括弧列は固定 seed と生成器で再現できる。
- 公開指標が比べるのは末尾要素であり、列全体の補完の正しさは、同じ出力を列全体でも照合して捉える。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py
[2] HELM v0.3.0. exact_match_indicator. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

Appendix

1. 追加の実例
2. 生成と採点の補足
3. 研究と参考文献

実例 4 ：角括弧を一つ閉じる

入力の全文／ 61 記号。空白で区切り、表示上の改行だけを追加。

```
[ ( ( [ [ ( ( { ( ) } ) ) { } ] { } { [ [
( ) ] ] } { [ [ [ [ ] ] ] ] } [ { { { { (
{ [ { ( ) } ] } [ ] ) } } } } ] ] ) )
```

正解：]　　予測：]

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =1 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id6 / test / generator index 6
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 5 ：丸括弧を一つ閉じる

入力の全文／ 77 記号。空白で区切り、表示上の改行だけを追加。

```
[ [ ] ] ( { } ) ( ( { { { { ( ( [ ( { }
) [ { } ] ] ) ) } } ( ) [ { [ [ ] ] ( {
[ ] } ) } ] [ { [ [ ] ] } ] [ { { } } ]
{ { ( [ ] { ( ) } ) } } ( ) } } )
```

正解：)　　予測：)

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =1 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id7 / test / generator index 7
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 6 ：波括弧を一つ閉じる

入力の全文／ 67 記号。空白で区切り、表示上の改行だけを追加。

```
( ( ) ) { [ ] { ( ( ( { [ ( ( ) ) ] { [ { ( )
} ] } [ [ ] ] ( ) [ [ { ( ( ( ( ( ) ) ) ) ) }
] [ ] { ( ) } { { } } ] ( ( ) ) } ) ) ) }
```

正解： }　　予測： }

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =1 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id8 / test / generator index 8
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 7 ： 98 記号の入力を補う

入力の全文／ 98 記号。空白で区切り、表示上の改行だけを追加。

```
( ) ( [ ] ) { } [ [ { { { ( ( ) ) } } [
( { { } } ) ] } ] ] ( [ ] ) [ ] ( [ [ [
{ { ( { [ { } ( ) ] } ) { [ [ ( ) ] ] }
} { { { [ ( { [ [ [ { { { } } } ] ] ] {
{ } } } ) { } ] } } } } ] ] { { } }
```

正解：])　　予測：)]

text-davinci-003 の試行 0 ：末尾要素の一致 =0 、列全体の一致 =0 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id10 / test / generator index 10
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 8 ：三つの括弧を逆順に閉じる

入力の全文／ 53 記号。空白で区切り、表示上の改行だけを追加。

```
{ ( { { [ { ( [ ] [ ] ) } ] } } ( [
] [ [ [ ( ) ] ( [ [ ( ( ( [ [ ] { (
) } ] ) ) ) ] [ ( ) ] ] ) ] ] ) [
```

正解：]) }　　予測：]) })

text-davinci-003 の試行 0 ：末尾要素の一致 =0 、列全体の一致 =0 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id11 / test / generator index 11
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 9 ：二つの括弧を閉じる

入力の全文／ 70 記号。空白で区切り、表示上の改行だけを追加。

```
[ ( ( [ [ ] ] ( ) ) { ( [ [ { ( ) } ( ( [ ( { [
[ ( { [ ( [ ( [ ( [ ] ) ] ) ( ) [ [ { { ( [ ] )
} } ] ] ] ) ] } ) ] ] } ) ] ) ) ( ) ] ] ) }
```

正解：)]　　予測：)]

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =1 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id12 / test / generator index 12
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 10 ：同じ種類が離れて現れる

入力の全文／ 91 記号。空白で区切り、表示上の改行だけを追加。

```
{ { } [ ] ( ) } [ ( [ [ { [ ] } ] ] ) ] [ ( {
( ( { { } } ) { } ) } ) [ { { { } } ( ( [ ( [
( { ( ) } ) ] ) ] [ ( ) ] ) ) { [ { } ] { [ ]
} { { } } } { } ( { } { [ ] } ( ) ) } ] ( [
```

正解：])]　　予測：])]

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =1 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id13 / test / generator index 13
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 11 ： 49 記号の入力を補う

入力の全文／ 49 記号。空白で区切り、表示上の改行だけを追加。

```
[ ( { ( [ { [ ( { [ [ [ ( ) ] ] ]
} ) ] } ] ) { ( ( ( { } ( ) ( ) (
( [ ( ) ] ) ) ) ) [ ( ) ] ) }
```

正解： })]　　予測： })]

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =1 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id14 / test / generator index 14
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 12 ：複数の種類が重なる

入力の全文／ 89 記号。空白で区切り、表示上の改行だけを追加。

```
[ { } ] [ ] ( { ( [ ] ) } [ ( { [ [ { [ ( [ [
] ] ) { ( ) } { [ [ ] ] } ] } ] ] } ) ] ( ) )
{ { [ ( { } ) ] } { } { [ ( ) ] } { } ( { { }
( ) ( ( ) ) } ) [ { [ ( { [ ] } ) { } ]
```

正解： `} ] }`　　予測：`} ] ]`

text-davinci-003 の試行 0 ：末尾要素の一致 =0 、列全体の一致 =0 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id15 / test / generator index 15
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

実例 13 ：二つの閉じ括弧が必要

入力の全文／ 74 記号。空白で区切り、表示上の改行だけを追加。

```
[ { ( { { { { } ( [ [ { } ] ] ) [ ] }
} } ( ( ( ) ) ) { [ ] } [ [ ( [ ] (
[ ( [ ] ) ] { { ( ( ) ) } } ) ) [ [ {
} ] ] ] [ ( ) ( ) ] ( ( ) ) ] } (
```

正解： ）]　　予測： ） ）]

text-davinci-003 の試行 0 ：末尾要素の一致 =1 、列全体の一致 =0 。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。 id16 / test / generator index 16
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003

公開プロンプトには完成した 3 列が先行する

- 公開 run の指示に続き、 3 件の提示例と、補完対象の入力が並ぶ。提示例では入力と正解の閉じ括弧が連続し、完成した列になる。
- 公開設定は max_train_instances=3 、 num_train_trials=3 。確認した 3 モデルは平均提示例数 3 、prompt_truncated=0 。
- 生成は温度 0 、最大 5 トークン、改行で停止。括弧記号数と、モデルの生成トークン数は異なる。

[1] HELM Classic v0.4.0, Dyck Language. 公開 run suite v0.3.0 、括弧 3 種類。 https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/dyck_language

長さの設定値と、切り出した入力を区別する

- 評価用の生成列を 52 〜 100 記号としても、入力の最小値が 52 記号になるわけではない。
- get_input は、後ろからの深さの合計が指定値に達する位置で列を切る。後ろにある対応済み部分列も切り落とすことがある。
- そのため、この seed では入力が 45 〜 99 記号、正解が 1 〜 3 記号になった。値は公開入力との全件照合に基づく。

[1] HELM v0.3.0. DyckLanguageScenario. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py

公開成績と独自の再採点を分けて保存する

- 3 モデルの末尾要素得点は、公開 stats.json の test ・摂動なしを参照した。全て 500 件、 3 試行。
- text-davinci-003 の列全体得点は、公開 scenario_state.json の 1,500 出力に、前後の空白を除く一致判定を適用した独自集計。
- 公開指標の実装も同じ出力へ適用し、試行 0 が 0.732 、試行 1 ・ 2 が各 0.760 、平均 0.750666… となることを再現した。

[1] HELM v0.3.0 、 seed=42 の生成器で再現し、公開 text-davinci-003 入力と全 500 件を照合。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/runs/dyck_language_np=3:model=openai_text-davinci-003
[2] HELM v0.3.0. exact_match_indicator. https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/basic_metrics.py

参考文献

Memory-Augmented Recurrent Neural Networks Can Learn Generalized Dyck Languages

Mirac Suzgun, Sebastian Gehrmann, Yonatan Belinkov, Stuart M. Shieber (2019). arXiv:1911.03329.

外部メモリを持つ再帰型モデルを提案し、 Dyck 言語や回文などで長い系列への汎化を調べる。

https://arxiv.org/abs/1911.03329

Dyck Language

Mirac Suzgun et al. Google BIG-bench, task documentation (2026-10-18 参照) 。

括弧列の欠けた末尾を選ぶ固定データと、生成文法や初期実験を掲載する。

https://github.com/google/BIG-bench/tree/main/bigbench/benchmark_tasks/dyck_languages

参考文献

Holistic Evaluation of Language Models

Percy Liang et al. (2023). Transactions on Machine Learning Research.

多様なシナリオと指標でモデルを評価する HELM の枠組みを提案する。

https://arxiv.org/abs/2211.09110

DyckLanguageScenario

Stanford CRFM (2023). HELM v0.3.0, Python implementation.

確率的文法による括弧列生成、入力の切り出し、スタックによる正解計算を実装する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/scenarios/dyck_language_scenario.py